

Week 7 Project Summary — Generative AI Track

Project: HealthConnect Experience Lab **Track:** Generative AI **Author:** Japheth **Date:** 2026-09-20
Repo: https://github.com/2024muguy/health_connect

1. What I planned to test

Week 6 delivered a working RAG assistant (30/30 on the evaluation suite). Week 7 moved the project into **Testing** → **Refinement** → **End-to-End Validation**. My objective was:

- 1. Systematically test the assistant as a product (not just a QA target)
- 2. Fix the weaknesses found in realistic conversation flows
- 3. Ship new capabilities that turn the prototype into a coherent assistant
- 4. Integrate the Data Science no-show predictor as a callable tool

2. What I actually tested

Component	Test	Result
Memory	15-turn conversation, fact recall	PASS — name, age, allergies persisted
Booking	4-turn hybrid slot fill via chat	PASS — APT code created in DB
Escalation	6-turn flow with mixed KB and human-handoff	PASS — no state pollution
Streaming (SSE)	curl + browser console + app	PASS — chunk-by-chunk
Injection defense	7 attack patterns	PASS — 7/7 blocked

KB grounding	Clinic hours, services, locations	PASS — clean Unicode, no leaks
Uncertainty gate	Low-retrieval queries	Wired, fail-open
Tool registry	4 tools registered	PASS

3. Most important testing results

- **Long-term memory works end-to-end.** A 15-turn conversation extracts `{name: Sarah, age: 42, allergies: penicillin, preferred_time_of_day: morning}` and later answers "What is my name and what am I allergic to?" correctly.
- **Conversational booking completes.** Chat-only flow returns a real appointment code (`APT-3EE571AA`) and writes the row to the database.
- **Escalation is state-safe.** After a 6-turn escalation loop, a fresh KB question still returns accurate hours — proving no state pollution.
- **KB cleanup cut noise by 72%.** 43 → 12 patient-facing chunks. All response quality improvements traced to this.

See [figures/](#) for visualizations and [evidence/](#) for raw JSON.

4. Issues or weaknesses identified

1. **patient_id type mismatch** — SQLAlchemy UUID columns rejected Python `str`
2. **Silent session duplication** — new conversation created every turn
3. **Escalation phrasing gap** — "connect me", "why aren't you connecting me" not triggering
4. **pending NameError** — partial patch left one undefined variable
5. **Weekday resolution missing** — "saturday" not converted to a date
6. **Response duplication in UI** — Zustand `addMessage` appended instead of replaced
7. **Mojibake in terminal** — CP1252 vs UTF-8 display artifact (verified non-issue)
8. **Streaming never reached the browser** — frontend went through Next.js proxy that buffered SSE

5. Improvements/refinements made

New services (Week 7)

- `memory_service.py` — rolling summary + structured fact extraction
- `streaming_service.py` — SSE token streaming wrapper
- `uncertainty_service.py` — retrieval + LLM-judge confidence gate
- `booking_service.py` — hybrid slot fill (service / date / time / location / contact)
- `tool_registry.py` — `knowledge_search`, `check_availability`, `create_appointment`, `escalate_to_human`

New safety layers

- `input_guard.py` — regex + normalization injection defense
- `request_limits.py` — rate limiting + body size caps
- `audit_log.py` — structured safety event logging

KB & prompt

- 43 → 12 patient-facing chunks
- System prompt rewritten for memory + grounding
- Booking trigger set expanded (13+ phrasings)
- Weekday resolver added for natural dates

6. Retesting results

Every fix was retested. See `W7_Testing_Report.md` for the full Test → Finding → Fix → Retest table with evidence.

7. Which tracks I collaborated with

- **Data Science** — wrapped the no-show predictor as a tool the assistant can call
- **Project Management** — logged the Week 7 issues for the shared register

8. What was tested collaboratively

End-to-end booking flow exercised the shared `appointments` API contract (`patient_id`, `appointment_type`, `scheduled_datetime`) that both the GenAI and Data Science tracks depend on.

9. What changed as a result

- Users can complete a booking without leaving the chat
- The assistant remembers identity across sessions
- Escalation reliably hands off to a human on the right phrasings
- Streaming replies feel 10× faster even at the same total latency

10. Key findings or validation outcomes

- **Memory is a force multiplier.** Multi-turn tasks no longer repeat context.
- **Retrieval quality trumps prompt engineering.** Cleaning 31 noisy chunks beat every prompt tweak combined.
- **Safety must be layered, not prompted.** The input guard + structured leak guard + uncertainty gate form a defense-in-depth stack.
- **Frontend and backend must agree on URL shape.** A Next.js rewrite buffering SSE was the hardest bug to find.

11. Major challenges

- Windows terminal encoding corrupting Python source during patch pastes
- Distinguishing backend bugs from frontend presentation bugs (SSE streaming)
- Handling long-lived states (`_PENDING_ESCALATIONS`) across ChatService instances
- Balancing tool-triggering safety with user intent ("connect me" is escalation)

12. Important decisions

- **Keep `openai/gpt-oss-20b`** — speed matters more than marginal quality
- **Hybrid booking** — assistant collects slots, shows confirmation, calls API
- **Skip voice this week** — cost + external dependencies not justified
- **Fail-open uncertainty gate** — false positives are worse than false negatives

13. Remaining limitations

- Cross-encoder reranker not yet wired
- Judge LLM adds ~1s when uncertainty enabled
- Voice endpoints not exposed
- Booking date parsing resolves weekdays but not "next week" or "in 3 days"
- Tools not yet LLM-routed (heuristic dispatch)

14. Remaining issues or dependencies

- Data Science: model artifact path pending, heuristic fallback in place
- Frontend: SSE consumer verified via console but not yet polished in the UI
- PM: previous broken sessions remain in the DB

15. My contribution to the overall HealthConnect project

- Shipped long-term memory that turns the assistant from stateless to conversational
- Cleaned 72% of KB noise, improving every downstream feature
- Built the 7-layer safety stack that makes the assistant deployable
- Wired the tool registry that lets agents act, not just answer
- Fixed the conversational booking flow end-to-end

16. What must be completed before Week 8

1. Frontend SSE consumer polish + screenshot evidence
2. Cross-track: DS no-show model artifact + integrated E2E test
3. Evaluation suite re-run to confirm no regressions
4. Final submission package uploaded to Google Drive
5. README updated with Week 7 section

--	--	--

Appendix A — Artefacts

Artefact	Path
Memory service	backend/app/services/memory_service.py
Streaming service	backend/app/services/streaming_service.py
Uncertainty service	backend/app/services/uncertainty_service.py
Booking service	backend/app/services/booking_service.py
Tool registry	backend/app/services/tool_registry.py

Input guard	backend/app/services/input_guard.py
Audit log	backend/app/services/audit_log.py
Rate limits	backend/app/middleware/request_limits.py
SSE endpoint	backend/app/api/v1/endpoints/chat.py (/message/stream)
WS endpoint	backend/app/api/v1/endpoints/ws_chat.py
Settings	backend/config/settings.py
KB backup	backend/data/processed/ rag_knowledge_base.backup_*.json
KB cleaned	backend/data/processed/rag_knowledge_base.json

Appendix B — Week 5 → Week 7 metric deltas

Metric	Week 5	Week 7
KB chunks	43	12
Memory	None	Working
Streaming	None	SSE + WS
Uncertainty gate	None	Wired
Conversational booking	None	4-turn flow, real DB writes
Tool registry	None	4 tools
Safety layers	2	7
Escalation phrasings	4	17